

Advanced Topics in Networking

CSE 322

At A Glance ...

ସଫଳତା ଆହାର device
ଶୃଙ୍ଖଳାତ communication କରନ୍ତୁ
ଟେମ୍ପୋରାଲ congestion
algorithm କିଛି? → Reno,
Westwood etc

- An advanced execution
- An advanced command
- An advanced tool

An Advanced Execution

- TCP Congestion Control Algorithms (CCAs) in Linux

– Source: <https://jasonmurray.org/posts/2021/tcpcca/>

- Check the current CCA *get tcp congestion control*

– sysctl net.ipv4.tcp_congestion_control *system control network*

Output Example:

```
jemurray@home-server:~$ sysctl net.ipv4.tcp_congestion_control
✓ net.ipv4.tcp_congestion_control = cubic
```

TCP CCA in Linux

- Check for the default CCAs in Ubuntu

– `sysctl net.ipv4.tcp_available_congestion_control` → available congestion control

Output Example:

```
jemurray@home-server:~$ sysctl net.ipv4.tcp_available_congestion_control
net.ipv4.tcp_available_congestion_control = reno cubic
```

→ available default congestion control

TCP CCA in Linux (contd.)

- List installed, but not necessarily loaded CCAs

→ tcp এর জন্য kernel model খুঁজা install করা আছে

— `ls -al /lib/modules/`uname -r`/kernel/net/ipv4/tcp*`

Output Example:

```
server:~$ ls -al /lib/modules/`uname -r`/kernel/net/ipv4/tcp*
-rw-r--r-- root 14113 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_bbr.ko
-rw-r--r-- root 10913 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_bic.ko
-rw-r--r-- root 13865 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_cdg.ko
-rw-r--r-- root 10041 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_dctcp.ko
-rw-r--r-- root  8401 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_diag.ko
-rw-r--r-- root  7489 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_highspeed.ko
-rw-r--r-- root  9985 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_htcp.ko
-rw-r--r-- root  8073 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_hybla.ko
-rw-r--r-- root  8985 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_illinois.ko
-rw-r--r-- root  8617 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_lp.ko
-rw-r--r-- root 12561 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_nv.ko
-rw-r--r-- root 13441 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_probe.ko
-rw-r--r-- root  6449 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_scalable.ko
-rw-r--r-- root 11073 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_vegas.ko
-rw-r--r-- root  7609 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_veno.ko
-rw-r--r-- root  8097 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_westwood.ko
-rw-r--r-- root  7945 May  7 19:44 /lib/modules/4.15.0-144-generic/kernel/net/ipv4/tcp_yeah.ko
```

যদি kernel শক্ত

TCP CCA in Linux (contd.)

- Use modprobe to load additional modules

– `sudo /sbin/modprobe tcp_bbr` *tcp-bbr টি Load হবে।*

Output Example:

```
jemurray@home-server:~$ sudo /sbin/modprobe tcp_bbr
```

```
jemurray@home-server:~$ sysctl net.ipv4.tcp_available_congestion_control  
net.ipv4.tcp_available_congestion_control = reno cubic bbr
```

*→ inoperation ন রাখলে
আমাদের ১০ আছে cubic
দেখাইছি তুমি এটা inoperation
প্রত্যাহা
} এই scene
in operation
একিছুই
reno cubic
bbr নাহে
নে। Load
বুঝার জন্য
দিয়ে।*

TCP CCA in Linux (contd.)

- Use sysctl to temporarily change the CCA to bbr
 - `sudo /sbin/sysctl -w net.ipv4.tcp_congestion_control=bbr`

```
jemurray@home-server:~$ sudo /sbin/sysctl -w net.ipv4.tcp_congestion_control=bbr
net.ipv4.tcp_congestion_control = bbr

jemurray@home-server:~$ sysctl net.ipv4.tcp_congestion_control
net.ipv4.tcp_congestion_control = bbr
```

TCP Code in Ubuntu

- Tentative location: `/net/ipv4/tcp.c`
- A snapshot of a part of the code ...

```
2734 tcp_call_bpf_2arg(sk, BPF_SOCKET_STATE_CB, oldstate, state);
2735
2736 switch (state) {
2737 case TCP_ESTABLISHED:
2738     if (oldstate != TCP_ESTABLISHED)
2739         TCP_INC_STATS(sock_net(sk), TCP_MIB_CURRESTAB);
2740     break;
2741
2742 case TCP_CLOSE:
2743     if (oldstate == TCP_CLOSE_WAIT || oldstate == TCP_ESTABLISHED)
2744         TCP_INC_STATS(sock_net(sk), TCP_MIB_ESTABRESETS);
2745
2746     sk->sk_prot->unhash(sk);
2747     if (inet_csk(sk)->icsk_bind_hash &&
2748         !(sk->sk_userlocks & SOCK_BINDPORT_LOCK))
2749         inet_put_port(sk);
2750     fallthrough;
2751 default:
2752     if (oldstate == TCP_ESTABLISHED)
2753         TCP_DEC_STATS(sock_net(sk), TCP_MIB_CURRESTAB);
2754 }
2755
2756 /* Change state AFTER socket is unhashed to avoid closed
2757  * socket sitting in hash tables.
2758  */
2759 inet_sk_state_store(sk, state);
2760 }
2761 EXPORT_SYMBOL_GPL(tcp_set_state);
2762
```

Linux এর একটি function

অন্যদিকে এখন google.com এ যাওয়ায় কন্সল থেকে ip খুঁজে পাব হলে যাব তা
জানা যায় (trace route google.com) এই command এর মাধ্যমে।

An Advanced Command

- traceroute
 - Determines the route to a node
 - tracert: Windows
 - traceroute: Unix

A Sample Output of “tracert”

↳ windows

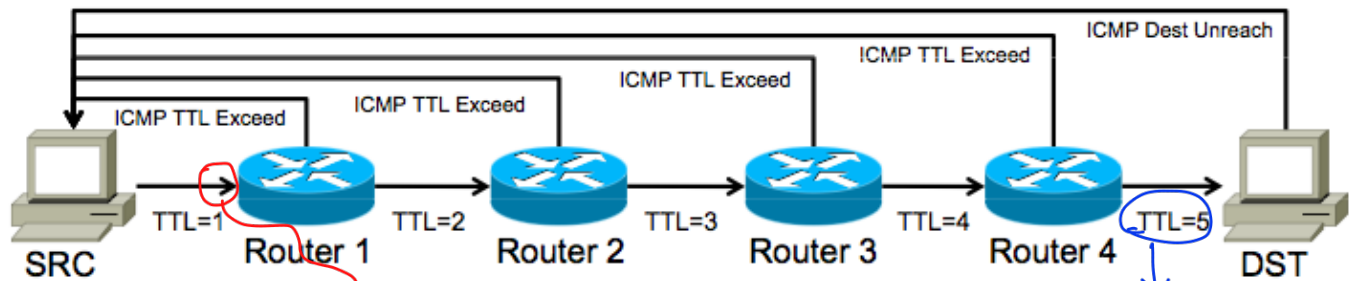
```
C:\Windows\system32\cmd.exe
C:\Users\User>tracert google.com

Tracing route to google.com [142.251.10.102]
over a maximum of 30 hops:
  1  3 ms  2 ms  1 ms  192.168.0.1 → Ip address
  2  10 ms  4 ms  2 ms  10.0.2.1
  3  5 ms  13 ms  2 ms  10.100.0.1
  4  44 ms  30 ms  3 ms  43.224.112.136
  5  4 ms  5 ms  6 ms  103.124.224.10
  6  52 ms  60 ms  50 ms  103.124.224.140
  7  *  *  *  Request timed out.
  8  *  *  51 ms  209.85.243.27
  9  635 ms  873 ms  120 ms  108.170.240.172
 10  *  *  *  Request timed out.
 11  54 ms  55 ms  *  216.239.41.225
 12  77 ms  *  *  142.251.52.67
 13  *  *  *  Request timed out.
 14  *  *  *  Request timed out.
 15  *  *  *  Request timed out.
 16  *  *  *  Request timed out.
 17  *  *  *  Request timed out.
 18  *  *  *  Request timed out.
 19  *  *  *  Request timed out.
 20  *  *  *  Request timed out.
 21  *  *  *  Request timed out.
 22  *  *  *  Request timed out.
 23  *  *  *  Request timed out.
 24  51 ms  58 ms  53 ms  sd-in-f102.1e100.net [142.251.10.102]

Trace complete.
C:\Users\User>
```

How Does Traceroute Work?

1. Launch a probe packet towards DST, with a TTL of 1
2. Every router hop decrements the IP TTL of the packet by 1
3. When the TTL hits 0, packet is dropped, router sends ICMP TTL Exceed packet to SRC with the original probe packet as payload
4. SRC receives this ICMP message, displays a traceroute "hop"
5. Repeat from step 1, with TTL incremented by 1 each time, until...
6. DST host receives probe, returns ICMP Dest Unreachable



src থেকে 1 টি packet
Router-1 এ আসলো
with TTL=1, then
Router 1 এ TTL
decrement করলো
TTL=0 হলো।

TTL 0 হলে আমরা
কে packet টি drop
করি আর Router
এ TTL 0 হলে (যখন Router 1)
তৈ router থেকে ICMP TTL
exceed packet পাঠানো
হয় SRC এর কাছে।

ICMP TTL exceed এ
বর্তমানের রুটিনিক
থেকে packet টি পাঠানো
এই জায়গায় IP address
টি Router 1 থেকে ICMP
TTL exceed এর মাধ্যমে SRC
এখানে এই যে back করবে
IP সোর্সের ইনবোর্স বিনো।

যখন আমরা SRC থেকে
packet টি পাঠানো
But TTL এর value 1
increment করে দিবে
আবার Router 2 এও
সেইভাবে TTL 0 হবে।
same process.

Traceroute → আমরা
মূল destination
দিয়ে intermediate
device গুলো আলাদা
সামনে present করে।

TTL=5 হলে DST তে
গেলো এখন দেখাচ্ছে
ICMP dest unreachable
আমাদের application এ
connect করতে
with address-port
শুধু আমাদের SRC
থেকে পাঠানো
যায় pseudo Random
number
high port থেকে যত
আমাদের রুটিনো
application থাকবে।
তাই SRC থেকে যখন
DST এ আসে তখন
আমাদের DST এর
এই port number টি শুধু
সার্ভার ICMP dest
unreachable ফেরায়।

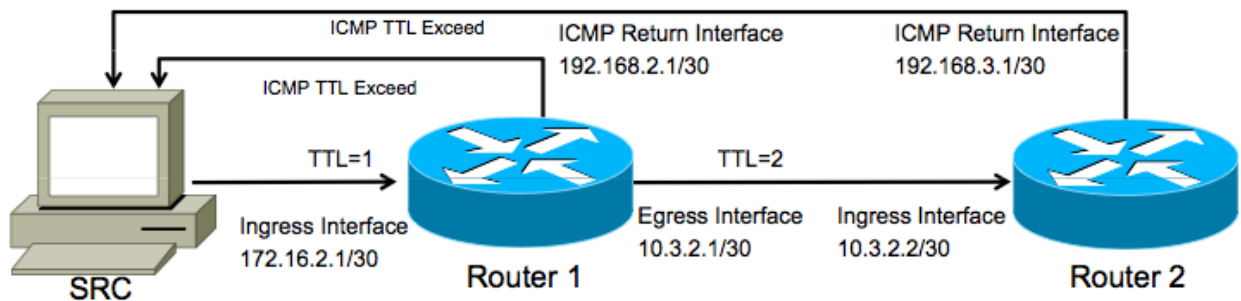
Responses in Traceroute

- All hops except the last one will (or should) return a "TTL exceeded in transit" message
- The last hop should return a "destination unreachable/port unreachable" message indicating that it cannot handle the received traffic
 - UDP traceroute packets are typically addressed to **a pseudorandom high port** on which the end host is not likely to be listening

Traceroute Report Hop

এখানে Time delay is both way, IP → one way।

- Traceroute packet with TTL of 1 enters router via the ingress interface
- Router decrements TTL to 0, drops packet, generates ICMP TTL Exceed
 - ICMP packet dst address is set to the original traceroute probe source (SRC)
 - ICMP packet src address is set to the IP of the **ingress** router interface
 - Traceroute shows a result based on the src address of the ICMP packet
 - The above traceroute will read: 172.16.2.1 10.3.2.2
 - You have **NO visibility into the return path or the egress interface** used



Traceroute Latency Calculation

- How is traceroute latency calculated?
 - Timestamp when the probe packet is launched
 - Timestamp when the ICMP response is received
 - Calculate the difference to determine round-trip time
 - Routers along the path do not do anytime “processing”
 - They simply reflect the original packet’s data back to the SRC
 - Many implementations encode the original launch timestamp into the probe packet, to increase accuracy and reduce state
 - Most Importantly: only the **ROUNDTrip** is measured
 - Traceroute is showing you the hops on the forward path *time is not roundtrip*
 - But showing you latency based on the forward **PLUS** reverse path *both reverse + forward path is time for sum of both*
 - Any delays on the reverse path will affect your results

An Advanced Tool

- Nmap
 - Source: <https://nmap.org/download.html#windows>
 - Extremely powerful
 - Extremely invasive
 - Extremely obvious if you are not careful
 - Extremely illegal if not done correctly



Port Scanning Basics

- nmap scans more than 1660 ports
- Most port scanners list ports as opened or closed
- nmap recognizes 6 port states
 - Open
 - Accepting TCP connections or UDP packets
 - Closed
 - Host is up on the IP address
 - Accessible but no app is listening
 - Try later

যেকোনো port scanner
হিসাবে open না close
এক বলা

Port Scanning Basics (contd.)

- nmap recognizes 6 port states (contd.)

Nmap 6254 67 categorization
on
2/12

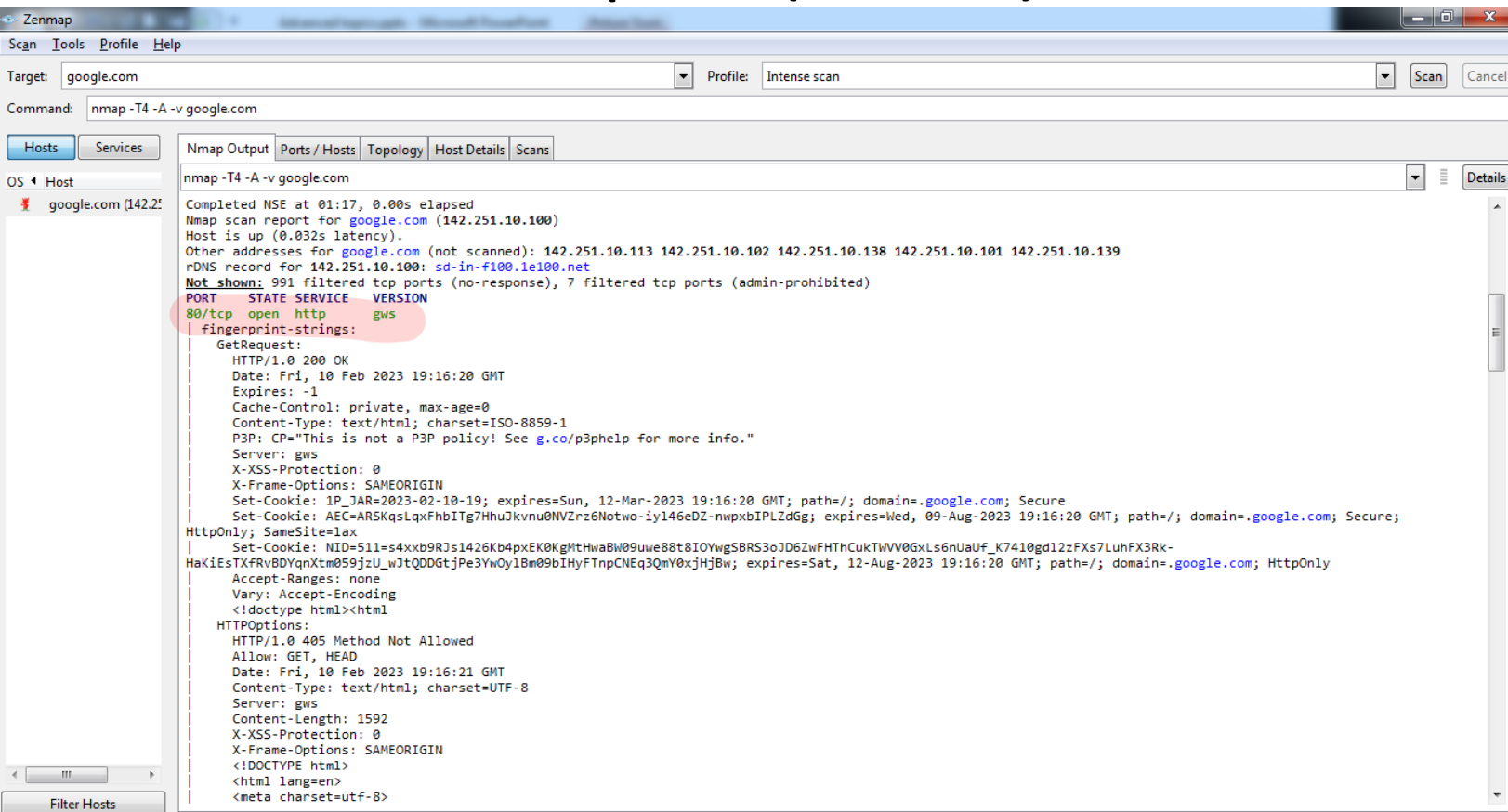
- Filtered
 - No response from probe
 - Firewall probably did a stealth drop
 - Forces nmap to retry many times
- Unfiltered
 - Port is accessible but cannot determine whether open or closed
- Open|Filtered
 - When unable to determine whether port is open or filtered
- Closed|Filtered
 - When unable to determine whether port is closed or filtered

An Intense Scan in nmap: “Nmap Output”

```
Starting Nmap 7.93 ( https://nmap.org ) at 2023-02-11 01:16 Bangladesh Standard Time
NSE: Loaded 155 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 01:16
Completed NSE at 01:16, 0.00s elapsed
Initiating NSE at 01:16
Completed NSE at 01:16, 0.00s elapsed
Initiating NSE at 01:16
Completed NSE at 01:16, 0.00s elapsed
Initiating Ping Scan at 01:16
Scanning google.com (142.251.10.100) [4 ports]
NSOCK ERROR [0.4810s] ssl_init_helper(): OpenSSL legacy provider failed to load.

Completed Ping Scan at 01:16, 2.42s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 01:16
Completed Parallel DNS resolution of 1 host. at 01:16, 0.05s elapsed
Initiating SYN Stealth Scan at 01:16
Scanning google.com (142.251.10.100) [1000 ports]
Discovered open port 443/tcp on 142.251.10.100
Discovered open port 80/tcp on 142.251.10.100
Completed SYN Stealth Scan at 01:16, 5.41s elapsed (1000 total ports)
Initiating Service scan at 01:16
Scanning 2 services on google.com (142.251.10.100)
Service scan Timing: About 50.00% done; ETC: 01:18 (0:01:01 remaining)
Completed Service scan at 01:17, 71.30s elapsed (2 services on 1 host)
Initiating OS detection (try #1) against google.com (142.251.10.100)
Retrying OS detection (try #2) against google.com (142.251.10.100)
Initiating Traceroute at 01:17
Completed Traceroute at 01:17, 4.08s elapsed
Initiating Parallel DNS resolution of 11 hosts. at 01:17
Completed Parallel DNS resolution of 11 hosts. at 01:17, 0.09s elapsed
NSE: Script scanning 142.251.10.100.
Initiating NSE at 01:17
Completed NSE at 01:17, 5.28s elapsed
Initiating NSE at 01:17
Completed NSE at 01:17, 6.61s elapsed
Initiating NSE at 01:17
Completed NSE at 01:17, 0.00s elapsed
Nmap scan report for google.com (142.251.10.100)
```

An Intense Scan in nmap : “Nmap Output” (contd.)



The screenshot shows the Zenmap interface with the target set to google.com and the profile set to Intense scan. The command is nmap -T4 -A -v google.com. The output is displayed in the 'Nmap Output' tab, showing the scan results for google.com (142.251.10.100). The output includes the Nmap scan report, host status, other addresses, rDNS record, filtered ports, and a detailed view of the open http service on port 80. The detailed view shows the HTTP request and response, including headers and body content.

```
nmap -T4 -A -v google.com

Completed NSE at 01:17, 0.00s elapsed
Nmap scan report for google.com (142.251.10.100)
Host is up (0.032s latency).
Other addresses for google.com (not scanned): 142.251.10.113 142.251.10.102 142.251.10.138 142.251.10.101 142.251.10.139
rDNS record for 142.251.10.100: sd-in-f100.1e100.net
Not shown: 991 filtered tcp ports (no-response), 7 filtered tcp ports (admin-prohibited)
PORT      STATE SERVICE VERSION
80/tcp    open  http    gws
| fingerprint-strings:
|_  GetRequest:
|_    HTTP/1.0 200 OK
|_    Date: Fri, 10 Feb 2023 19:16:20 GMT
|_    Expires: -1
|_    Cache-Control: private, max-age=0
|_    Content-Type: text/html; charset=ISO-8859-1
|_    P3P: CP="This is not a P3P policy! See g.co/p3phelp for more info."
|_    Server: gws
|_    X-XSS-Protection: 0
|_    X-Frame-Options: SAMEORIGIN
|_    Set-Cookie: 1P_JAR=2023-02-10-19; expires=Sun, 12-Mar-2023 19:16:20 GMT; path=/; domain=.google.com; Secure
|_    Set-Cookie: AEC=ARSKqsLqxHbITg7HhuJkvnu0NVZrz6Notwo-iy146eDZ-nwpxbIPLZdGg; expires=Wed, 09-Aug-2023 19:16:20 GMT; path=/; domain=.google.com; Secure; HttpOnly; SameSite=lax
|_    Set-Cookie: NID=511=s4xxb9RJs1426Kb4pxEK0KgMtHwaBw09uwe88t8IOYwgSBR53oJD6ZwFHTuCukTwwV0GxLs6nUaUf_K7410gd12zFXs7LuhFX3Rk-HaKiEsTXFrVBDYqnXtm059jzU_wJtQDDGtjPe3YwOylBm09bIHvFTnpCNEq3QmY0xjHjBw; expires=Sat, 12-Aug-2023 19:16:20 GMT; path=/; domain=.google.com; HttpOnly
|_    Accept-Ranges: none
|_    Vary: Accept-Encoding
|_    <!doctype html><html
|_    HTTPOptions:
|_    HTTP/1.0 405 Method Not Allowed
|_    Allow: GET, HEAD
|_    Date: Fri, 10 Feb 2023 19:16:21 GMT
|_    Content-Type: text/html; charset=UTF-8
|_    Server: gws
|_    Content-Length: 1592
|_    X-XSS-Protection: 0
|_    X-Frame-Options: SAMEORIGIN
|_    <!DOCTYPE html>
|_    <html lang=en>
|_    <meta charset=utf-8>
```

An Intense Scan in nmap : “Nmap Output” (contd.)

The screenshot shows the Zenmap application window. The 'Target' field is set to 'google.com' and the 'Profile' is 'Intense scan'. The command entered is 'nmap -T4 -A -v google.com'. The 'Nmap Output' tab is selected, displaying the following text:

```
nmap -T4 -A -v google.com

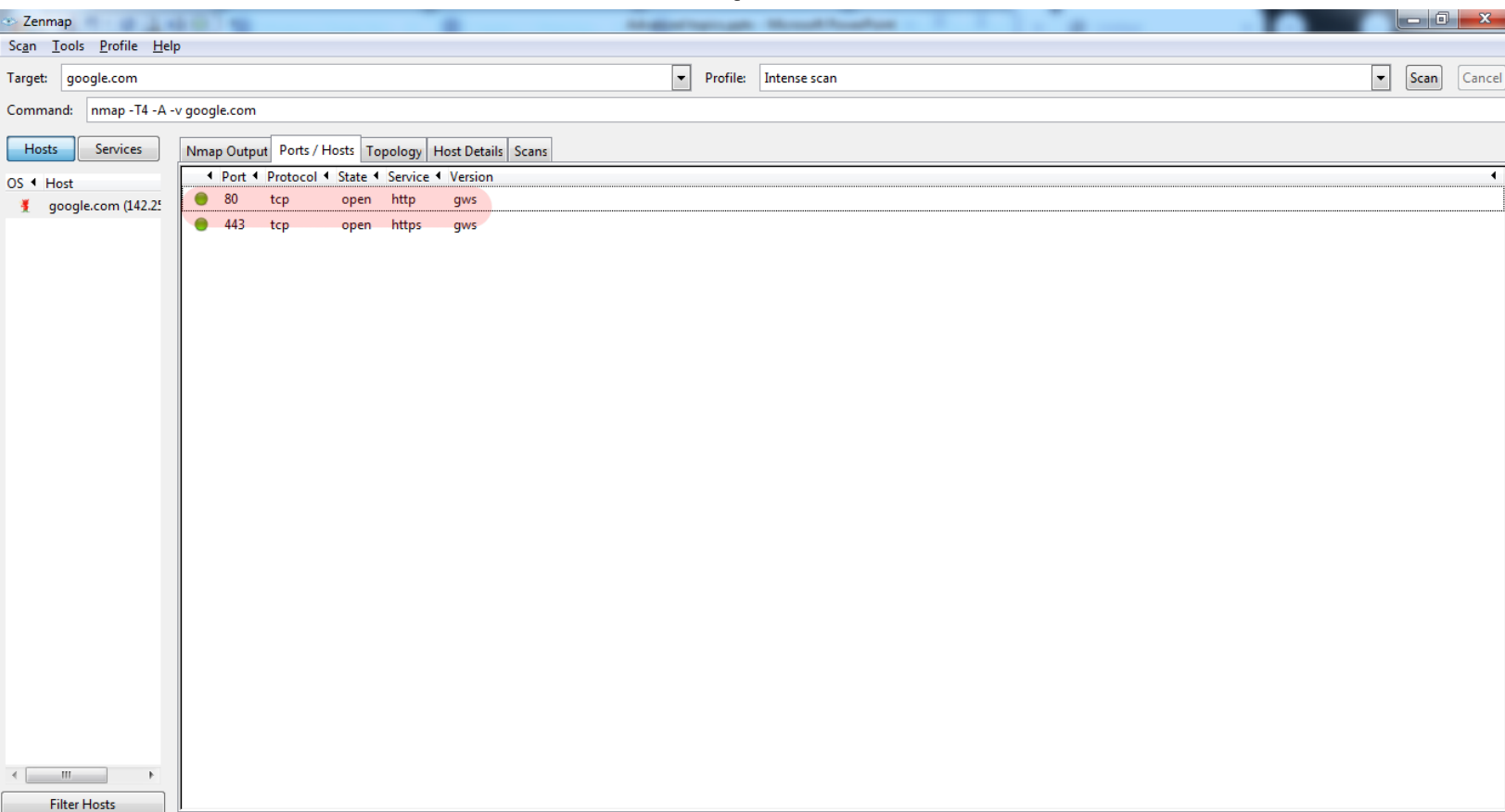
Device type: general purpose
Running (JUST GUESSING): FreeBSD 7.X (88%)
OS CPE: cpe:/o:freebsd:freebsd:7.0
Aggressive OS guesses: FreeBSD 7.0-STABLE (88%)
No exact OS matches for host (test conditions non-ideal).
Uptime guess: 0.000 days (since Sat Feb 11 01:17:31 2023)
Network Distance: 22 hops
TCP Sequence Prediction: Difficulty=262 (Good luck!)
IP ID Sequence Generation: All zeros

TRACEROUTE (using port 443/tcp)
HOP RTT ADDRESS
1 8.00 ms 192.168.0.1
2 8.00 ms 10.0.2.1
3 8.00 ms 10.100.0.1
4 8.00 ms 43.224.112.136
5 5.00 ms 103.124.224.10
6 58.00 ms 103.124.224.140
7 96.00 ms 72.14.205.188
8 49.00 ms 209.85.243.57
9 53.00 ms 108.170.240.164
10 53.00 ms 142.251.230.215
11 ...
12 51.00 ms 142.251.51.213
13 ... 21
22 52.00 ms sd-in-f100.1e100.net (142.251.10.100)

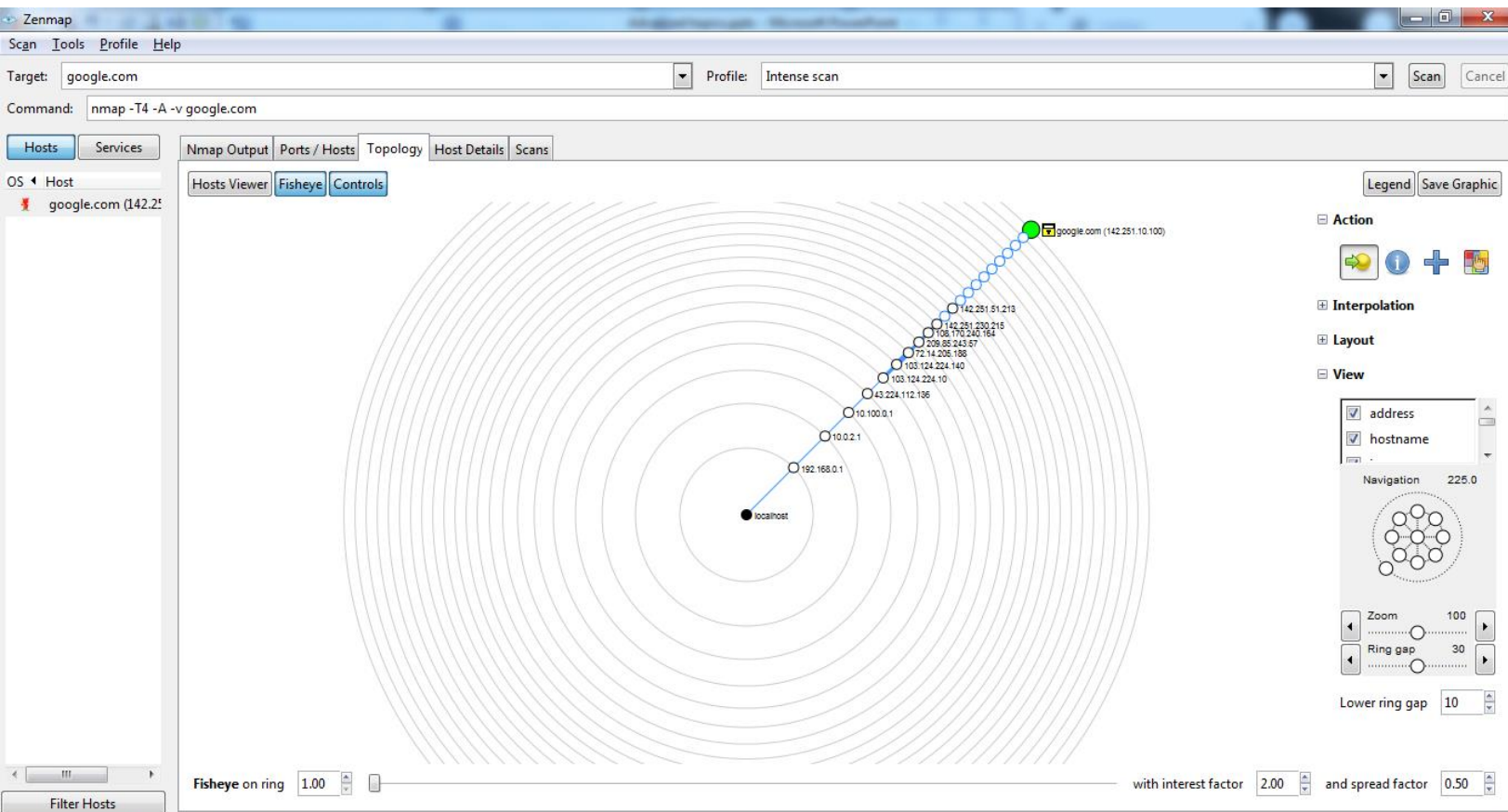
NSE: Script Post-scanning.
Initiating NSE at 01:17
Completed NSE at 01:17, 0.00s elapsed
Initiating NSE at 01:17
Completed NSE at 01:17, 0.00s elapsed
Initiating NSE at 01:17
Completed NSE at 01:17, 0.00s elapsed
Read data files from: C:\Program Files (x86)\Nmap
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 100.89 seconds
Raw packets sent: 2127 (98.226KB) | Rcvd: 85 (4.634KB)
```

The interface also shows a list of hosts on the left, with 'google.com (142.251.10.100)' selected. A 'Filter Hosts' button is visible at the bottom left.

An Intense Scan in nmap: “Ports/Hosts”



An Intense Scan in nmap: “Topology”



An Intense Scan in nmap: “Host Details”

Zenmap interface showing Host Details for google.com (142.251.10.100).

Target: google.com Profile: Intense scan Command: nmap -T4 -A -v google.com

Hosts Services Nmap Output Ports / Hosts Topology Host Details Scans

OS Host google.com (142.251.10.100)

Host Status

- State: up
- Open ports: 2
- Filtered ports: 998 → 998 पोर्ट फिल्टर हो गए हैं। 1000 पोर्ट स्कैन किए गए हैं।
- Closed ports: 0
- Scanned ports: 1000
- Up time: 18
- Last boot: Sat Feb 11 01:17:31 2023

Addresses

- IPv4: 142.251.10.100
- IPv6: Not available
- MAC: Not available

Hostnames

- Name - Type: google.com - user
- Name - Type: sd-in-f100.1e100.net - PTR

Operating System

- Name: FreeBSD 7.0-STABLE
- Accuracy: 88%

Ports used

OS Classes

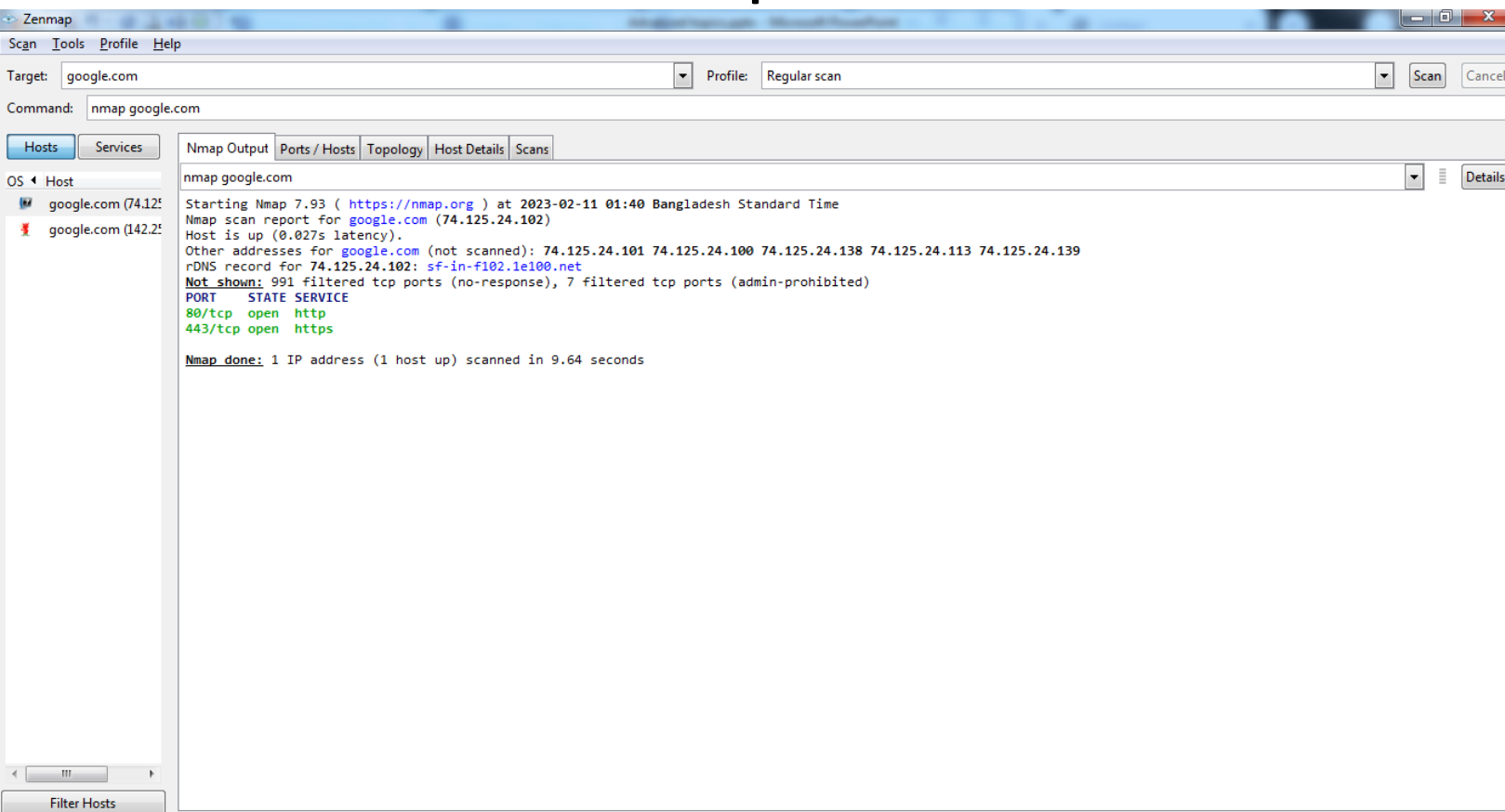
TCP Sequence

IP ID Sequence

TCP TS Sequence

port filter काटने का मतलब है ⇒ वहां तो firewall था कि वहां तो port अब info extract कराइ अगर firewall जगह से था कि

A Regular Scan in nmap: “Nmap Output”



Thank You